

SQL JOINS



What is SQL Join?

- ❖ JOIN clause combines rows from two or more tables.
- ❖ creates a set of rows in a temporary table.

Types of SQL JOIN

EQUI JOIN

EQUI JOIN is a simple SQL join.

Uses the equal sign(=) as the comparison operator for the condition NON EQUI JOIN

NON EQUI JOIN uses comparison operator other than the equal sign.

The operators uses like >, <, >=, <= with the condition.



Types of SQL EQUI JOIN

INNER JOIN

Returns only matched rows from the participating tables.

Match happened only at the key record of participating tables.

OUTER JOIN

Returns all rows from one table and

Matching rows from the secondary table and

Comparison columns should be equal in both the tables. List of SQL JOINS

INNER JOIN

LEFT JOIN OR LEFT OUTER JOIN

RIGHT JOIN OR RIGHT OUTER JOIN

FULL OUTER JOIN

NATURAL JOIN

CROSS JOIN

SELF JOIN

INNER JOIN

The INNER JOIN selects all rows from both participating tables as long as there is a match between the columns.

An SQL INNER JOIN is same as JOIN clause, combining rows from two or more tables.

Example: INNER JOIN

```
SELECT * FROM table_A
```

```
INNER JOIN table_B
```

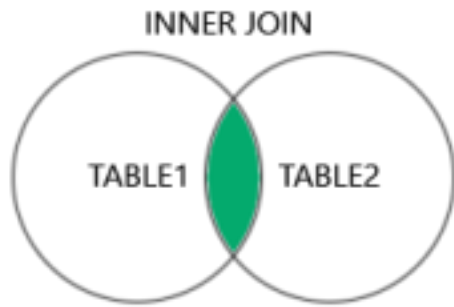


```
ON table_A.A=table_B.A;
```

The INNER JOIN keyword selects records that have matching values in both tables.

Products

ProductID	Product Name	CategoryID	Price	ProductID	Product Name	CategoryID	Price
1	Chais	1	18	2	Chang	1	19
				3	Aniseed Syrup	2	10



CategoryID	CategoryName	Description
1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
3	Confections	Desserts, candies, and sweet breads

Categories

```
SELECT ProductID, ProductName, CategoryName  
FROM Products
```

INNER JOIN Categories ON Products.CategoryID =
Categories.CategoryID;

JOIN and INNER JOIN will return the same result.

SELECT Products.ProductID, Products.ProductName,
Categories.CategoryName FROM Products

JOIN Categories ON Products.CategoryID =

Categories.CategoryID;



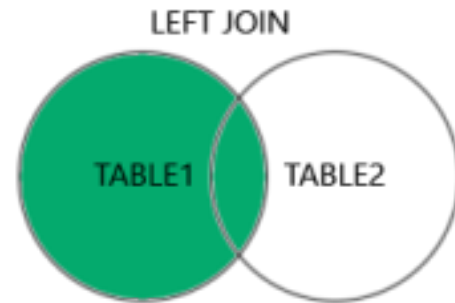
LEFT JOIN or LEFT OUTER JOIN

The LEFT JOIN keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

The unmatched rows will also be available from the table before the JOIN clause.

Example: LEFT JOIN or LEFT OUTER JOIN

```
SELECT * FROM table_A  
LEFT JOIN table_B  
ON table_A.A=table_B.A;
```



RIGHT JOIN or RIGHT OUTER JOIN

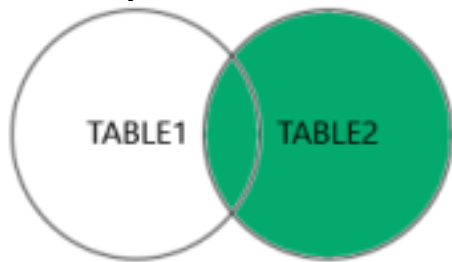
RIGHT JOIN

The RIGHT JOIN keyword returns all records from the right table (table2), and the matching records from the left table (table1).

The result is 0 records from the left side, if there is no match.

The unmatched rows will also be available from the table written after the JOIN clause.

Example: RIGHT JOIN or RIGHT OUTER JOIN



```
SELECT * FROM table_A
```

```
RIGHT JOIN table_B
```

```
ON table_A.A=table_B.A;
```




OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	9-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

EmployeeID	LastName	FirstName	BirthDate	
1	Davolio	Nancy	12/8/1968	Employee
2	Fuller	Andrew		952 .pic
3	Leverling	Janet		963 .pic

```

SELECT Orders.OrderID, Employees.LastName,
Employees.FirstName FROM Orders
RIGHT JOIN Employees ON Orders.EmployeeID =
Employees.EmployeeID ORDER BY Orders.OrderID;

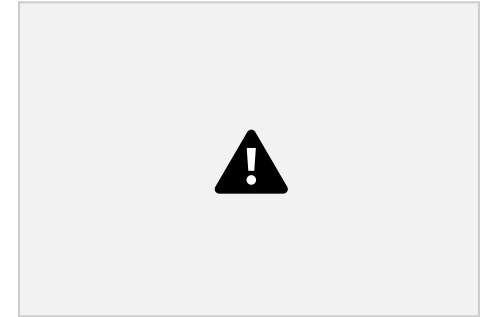
```



FULL OUTER JOIN

The FULL OUTER JOIN keyword returns all records when there is a match in left (table1) or right (table2) table records.

Example: FULL OUTER JOIN



```
SELECT * FROM table_A  
FULL OUTER JOIN table_B
```

```
ON table_A.A=table_B.A;
```

```
SELECT Customers.CustomerName, Orders.OrderID FROM  
Customers  
FULL OUTER JOIN Orders ON  
Customers.CustomerID=Orders.CustomerID ORDER BY
```

Customers.CustomerName;



CustomerName	OrderID
Null	10309
Null	10310
Alfreds Futterkiste	Null
Ana Trujillo Emparedados y helados	10308
Antonio Moreno Taquería	Null

SELF JOIN

A self join is a regular join, but the table is joined with itself. A self join is a join in which a table is joined with itself (Unary

relationships), specially when the table has a FOREIGN KEY which references its own PRIMARY KEY.

To join a table itself means that each row of the table is combined with itself and with every other row of the table.

The self join can be viewed as a join of two copies of the same

table. Example : SELF JOIN

```
SELECT *
```

```
FROM table_A X, table_A Y
```

```
WHERE X.A=Y.A;
```



CustomerID	Contact Name	PostalCode
1	Alfreds Futterkiste	Germany

2 Ana Trujillo Emparedados y helados	Ana Trujillo	2222 Mataderos 2312 México D.F.	05021 Mexico	05023 Mexico
3 Antonio Moreno Taquería	Antonio Moreno Avda. de la Constitución	México D.F.		

```
SELECT A.CustomerName AS CustomerName1, B.CustomerName
AS CustomerName2, A.City
FROM Customers A, Customers B
WHERE A.CustomerID <> B.CustomerID
AND A.City = B.City
ORDER BY A.City;
```



NATURAL JOIN

The SQL NATURAL JOIN is a type of EQUI JOIN and is structured in such a way that, columns with same name of associate tables will appear once only. The associated tables have one or more

pairs of identically named columns. The columns must be the same data type.

Don't use ON clause in a natural join.

Example: NATURAL JOIN

```
SELECT *  
FROM table_A  
NATURAL JOIN table_B;
```



CROSS JOIN

The SQL CROSS JOIN produces a result set which is the number of

rows in the first table multiplied by the number of rows in the second table, if no WHERE clause is used along with CROSS JOIN.

This kind of result is called as Cartesian Product.

If, WHERE clause is used with CROSS JOIN, it functions like an

INNER JOIN. Example: CROSS JOIN

```
SELECT *
```

```
FROM table_A
```

```
CROSS JOIN table_B;
```



The SQL BETWEEN Operator

The BETWEEN operator selects values within a given range. The

values can be numbers, text, or dates.

Selects all products with a price between 10 and 20:

```
SELECT * FROM Products  
WHERE Price BETWEEN 10 AND 20;
```

```
SELECT * FROM Products  
WHERE Price BETWEEN 10 AND 20  
AND CategoryID IN (1,2,3);
```



BETWEEN Text Values

```
SELECT * FROM Products
```

WHERE ProductName BETWEEN 'Carnarvon Tigers' AND
'Mozzarella di Giovanni'

ORDER BY ProductName;

NOT BETWEEN Text Values

SELECT * FROM Products

WHERE ProductName NOT BETWEEN 'Carnarvon
Tigers' AND 'Mozzarella di Giovanni'

ORDER BY ProductName;

BETWEEN Dates

SELECT * FROM Orders



WHERE OrderDate BETWEEN
#07/01/1996# AND #07/31/1996#;

